COMP101
Introduction to Programming
2025-26

Lecture-10

Selection Python

Terminology

- Python selection syntax uses three commands

if: … elif: … else: #a colon must follow if: else: elif:

- All code after the colon automatically indents
  - Default is a 4 space-indent
  - Best not to change it – gets messy!

- Python does not use
  - ‘then’
  - ‘case’ or ‘switch’
  - { } ‘delimeters’

Selection Syntax

- if - elif - else
- Normally one ‘if’ at start and one ‘else’ at the end
- As many ‘elif’ in between these as needed
- Test returns True or False
- Comparison (binary) operators can be used in test

  < >= <= > <> or (!=) ==
- Boolean operators can be used in test
  - and, or ,not
- break (used with iteration)
- continue (used with iteration)

‘equal to’
Evaluates True or False
1 == 2 evaluates F
2 == 2 evaluates T
a == a evaluates T

if statement: mechanics

```
#user throws a dice:  throw again if number is a 6
number = int(input("Enter a throw of a die, 1 to 6: "))  #sequence statement – left aligned

if (number == 6):                #sequence statement – left aligned but starts with 'if' so
                                 Python expects selection construct to follow it
    print ("Throw again")        #selection statement – 4-space indented under 'if'

print ("End of game")            # end of if-statement – this is a sequence statement – left aligned
```

- Usual to have user input before the if-statement
- if-statement performs a test on the user input
- The test must evaluate True or False (T/F)
  - we use binary operators for this

This one has no elif or an else in it – it is a 'short-form if'.
With elif or else in it it is a 'long-form if'

if: test = True

```
num = int (input ("Enter a number greater than 0: ")) 5

if (num > 0):                          #‘condition line’ – use a colon
    print (“Test is True. Number is positive")

print (“Selection exited – execution must continue at first un-indented code”)
```

```
Test is True. Number is positive
Selection exited – execution must continue at first un-indented code
```

When the test is True, all indented code after the selection always executes
When the test is False, ignore indented code under the if statement

If the second print statement were indented under the first print statement, it would be part of the test=True indentation and would print out both statements

if: test = True execute multiple statements

```
num = int(input("Enter a number greater than 0: ")) 5

if (num > 0):        # 5 > 0 = TRUE - execute all indented lines under the if
    print ("Test is True. Number is positive")
    print ("Print this line as well – it is indented under the if")

print ("Selection exited – execution continue here at first un-indented code")
```

```
Test is True.  Number is positive
Print this line as well – it is indented under the if
Selection exited – execution continue here at first un-indented code
```

if: test = False

```
num = int(input("Enter a number greater than 0: ")) -10

if (num > 0):        # -10 > 0 = FALSE, ignore all indented statements
    print ("Test is True. Number is positive")
    print ("Print this line also – it is indented under the if")

print ("Selection exited – execution continues at first un-indented code")
```

Selection exited – execution carries on at first un-indented code

if ... else: Extends previous slide to handle Test=False

Important: else: aligns with if, note colon after else

```
num = int(input("Enter a number greater than 0: ")) -10

if (num > 0):                                   #if executes when test = True
    print (“Test is True.  Number is positive") #scope is two statements
    print (“Print this line as well – it is indented under the if”)
else:                                           # else executes when test = False
    print (“Test is False.  Number is negative”) #scope is two statements
    print (“Print this line as well – it is indented under the else”)

print (“Selection exited – execution continues at first un-indented code”)
```

```
Test is False. Number is negative
Print this line as well – it is indented under the else
Selection exited – execution continues at first un-indented code
```

Correct test?

```
num = int(input("Enter a number greater than 0: ")) 0

if (num > 0):          #test = False (0 is not greater than 0 – ignore if and execute else)
   print ("Test is True.  Number is positive")  #scope simplified to one statement
else:
   print ("Test is False. Number is negative") #scope simplified to one statement

print ("Selection exited – execution continues at first un-indented code")
```

```
Test is False. Number is negative
Selection exited – execution continues at first un-indented code
```

- Problem: 0 is positive, not negative
- Syntactically the statement works, logically it doesn't

Don't panic! Check the input against the test condition

Correct test? Two solutions

Solution-1: Change the test

```
num = int(input("Enter a number greater than 0: ")) 0
if (num >= 0):                                  #num>0 is now num>=0
   print ("Test is True.  Number is positive or 0")
else:                                           # else executes when num < 0
   print ("Test is False. Number is negative")
print ("Selection exited – execution continues at first un-indented code")
```

```
Test is true. Number is positive or 0
Selection exited – execution continues at first un-indented code
```

Problem: Output leaves some confusion
If (num >=0):
Test is now True for input of 0 … but we don't know whether input=0 or input=positive integer

Correct test? Two solutions

- Solution-1 analysis:
  - Change the test from (num>0) to (num>=0)
  - if executes for input of 0 or for positive numbers
    - but it says ‘number is positive or 0’ – but which is it?
  - We prompt for input of positive integer
    - but no validation allows input of negative numbers
      - else executes correctly for negative numbers

- Solution-2:
- Use an ‘elif’ for stronger programming
- Allows us to use the else statement to catch ‘exceptions’

elif (test):

- elif: statements come between the if statement and the else statement and they are left aligned with if and else
- There can be as many ‘elif’ as needed to solve the problem
- elif statements need a test condition (like the if statement) – the test must be different from the if test

elif (test):

```
if (test):
    if statements
elif (test):
    elif statements
elif (test):
    elif statements
elif (test):
    elif statements
else:                          #no test needed – else 'captures the exception'
    else statements
```

if (test): elif (test): else: ‘catching the exception’

- A good structure using if – elif – else makes the program more robust and secure
- We look for one if statement with a test condition and one else statement (with no test condition). In between as many elif statements as are needed each with its own different test from the if and from each other
- With sensible logic we can use tests to eliminate inputs until only one input remains – this is the ‘exception’ and is caught by else:

elif: and ‘catching the exception’

```
#user does not follow instruction – look at the input. No validation
num = int(input("Enter a number greater than 0: ")) 0

if num > 0:                          #test is back to (num>0) and not (num>=0)
      print ("Test is True.  Number is positive")

elif (num < 0):                      #elif must have its own test
      print ("Test is False.  Number is negative")

else:                                #else does not need a test – only one answer remaining
      print (“Print this line when all tests are False. Input must be 0”)
print ("Execution carries on here - at next un-indented statement")
```

if tests for num>0	#one if statement – left aligned
elif tests for num<0	#as many as needed – all left aligned
else ‘catches the exception’	#one else statement – left aligned

When ‘if’ has not executed and ‘elif’ has not executed there’s nothing left to test, hence num must = 0 as there is no other option remaining

Logic dictates the final structure

- We don’t have to use all three elements
  - May only need an if
  - May only need an if…else
  - May need if…elif…else

- Good constructs with three elements
  - Only one ‘if’ at the start (must have a test)
  - Only one ‘else’ at the end (may not need a test)
  - Zero or more ‘elifs’ (must have a test) in between

Summary

- if test=T
  - execute all indented code beneath the ‘if’
  - exit the construct and find first un-indented code that is outside the construct: i.e. ignore any ‘elif’ or ‘else’
  - execute first un-indented code outside of construct
- If test = F
  - ignore all indented code under the ‘if’
  - find first un-indented code below the ‘if’
  - when first un-indented code is ‘elif’
    - execute it when the test is True otherwise ignore it and find next un-indented code below it (this might be another ‘elif’
  - When next un-indented code is ‘else’ execute it and exit

Summary

- Make the selection statement come immediately after the input line where user inputs data to be tested
- Don’t strictly need brackets around the test
  - good for readability
- In all cases, on exit from the selection construct, we WILL execute the first line of un-indented code outside of the construct

Summary

if: 'shortform if'	if … else 'longform if'

x = int(input("Enter an integer: "))
if (x < 0):
  print ("Number is negative")
	x = int(input("Enter an integer: "))
	if (x < 0):
	  print ("Number is negative")
	else:
	  print ("Number is not negative")

If …elif …else 'longform if'	Nested if 'longform if'

x = int(input("Enter an integer: "))
if (x < 0):
  print ("Number is negative")
elif (x > 0):
  print ("Number is positive")
else:
  print ("Number is zero")
Do the longform it this way
	x = int(input("Enter an integer: "))
	if (x < 0):
	  print ("Number is negative")
	else:
	  if (x > 0):
	  print("Number is positive")
	  else:
	  print ("Number is zero")

Summary - shortform if test = True

```
name = input ("Enter your given name: ")
print ("Hello",name, "Your password is 1234")
confirm_pwd = input ("Enter your given password to confirm receipt: ")1234

if confirm_pwd == '1234':                #this line is sequential code – it 'lines up' with the code above it
    print ("Password confirmed")
    print ("Welcome to the system")
print ("You entered the correct password - your system credentials are being processed")
```

Two lines of indented selection code ('children of the selection construct').
We can execute as many lines of indented code as needed

When the last indented selection statement executes, control returns to the first un-indented line below it. This un-indented line of code 'lines up' with the other sequential statements above it, hence it shows the selection statement has ended.
As this is now a sequence statement, it MUST execute as it is not part of the selection construct because it is not indented.

Summary - shortform if test = False

```
name = input ("Enter your given name: ")
print ("Hello",name, "Your password is 1234")
confirm_pwd = input ("Enter your given password to confirm receipt: ") 5678

if confirm_pwd == '1234':
    print ("Password confirmed")
    print ("Welcome to the system")

print ("You entered the correct password - your system credentials are being processed")
```

If the user DOES NOT enter 1234 the test evaluates False (not True)
"You entered the correct password – your system credentials are being processed"

Input of 5678 is not 1234 so all indented code of the if statement is ignored

You can see the problem we have to manage here – the last un-indented line of code MUST execute – but it's not what we want to say.

Summary - shortform if test = False

```
name = input ("Enter your given name: ")
print ("Hello",name, "Your password is 1234")
confirm_pwd = input ("Enter your given password to confirm receipt: ") 5678

if confirm_pwd == '1234':
    print ("Password confirmed")
    print ("Welcome to the system")

print (“Processing complete")
```

We could change the message in the final sequence statement:
“Processing complete”
This works ‘okay’(?) if test = True and the indented code executes.

But if the test = False (user does NOT input 1234) the indented code under the if does not execute and user receives message “Processing complete”.
User has no idea that what they entered was incorrect.
Solution: use an ‘else’ statement.

Summary - longform if (if … else)

```
name = input ("Enter your given name: ")
print ("Hello",name, "Your password is 1234")
confirm_pwd = input ("Enter your given password to confirm receipt: ") 5678

if confirm_pwd == '1234':
    print ("Password confirmed")
    print ("Welcome to the system")
else:
    print ("Password is incorrect")
    print (“CPD will mail you")

print("If you received confirmation, your system credentials are being processed")
print ("If you entered a wrong password, admin will mail you")
print (“Processing complete")
```

if = TRUE so execute indented code when user inputs 1234

if = False so ignore indented code under if and execute else part; else executes its indented code when inputs is not 1234

REMEMBER: It is better programming to use if … elif … else where the else part is used to ‘capture the exception’. We only have one ‘if’ and one ‘else’ and as many ‘elif’ as we need each with their own test

Selection: Construction errors

- Wrong syntax
  - Spell if, elif, else correctly with tests and colons
  - Check alignment and indentation
- Wrong tests
  - Especially Boolean for and/or/not and operators > < etc
- Wrong messages
  - Maybe the right message but in the wrong place
- Weak logic
  - Not using a robust structure
    - one if, one else, zero or more elif